Implementation Plan

Sparse, Low-Rank, and Deep: Music Source Separation
ENGS 109 (Spring 2026) Companion Document

Taka Khoo
May 30, 2026

Purpose. The operational sibling of the proposal. It answers *how* the project gets built, day by day, with playable audio at every checkpoint. Stage A already runs end-to-end (see `audio_demos/out/day0_*.wav`); the rest of this document is the path from there to the final report.

1 Sprint at a glance

Each row below produces *playable audio* that gets saved into `audio_demos/out/` and gets a row in the final report's audio-examples table. No deliverable is purely abstract.

Day	What I build	Course tie	Audio output
0 (done)	Inexact-ALM Robust PCA solver, applied to MODULO mix	L13 RPCA (P_{R_1})	`day0_{mix,harmonic,percussive}.wav`
1	MUSDB18-HQ loader; render mix + 4 stems for first 5 train songs	L1 motivation, L3 sampling	`day1_musdb_*.wav` (15 files)
2	Stage A applied to MUSDB; museval SDR baseline number	L13 + evaluation harness	`day2_stageA_song{1..5}.wav`
3	Train $\boldsymbol{D}_v, \boldsymbol{D}_i$ NMF dictionaries on 20-song subset	L15 Lee-Seung multiplicative updates	`day3_nmf_atoms.pdf` (visual atom catalog)
4	Stage B inference on 5 test songs; per-source WAVs	L15 separation via concatenated dictionary	`day4_stageB_*.wav` (10 files)
5	K-SVD dictionaries per source, OMP-based code	L14 K-SVD, PS3 OMP reused	`day5_ksvd_atoms.pdf`
6	Stage C SRC inference; produce binary masks	L8 SRC residual rule	`day6_stageC_*.wav`
7-10	SparseNet skeleton in PyTorch, overfit one song	L16 composite module + L10 FISTA backward	`day10_overfit_one_song.wav`
11-18	Full SparseNet training on MUSDB18-HQ train split	L16 end-to-end backprop	`day18_stageD_test_set.zip`
19-22	MMV stereo refinement (SOMP); add joint-sparsity post-pass	L12 simultaneous OMP	`day22_stageE_stereo_*.wav`
23-28	Full evaluation harness vs Demucs, Spleeter, ElevenLabs; ablations on K, S, λ	L6/L7 theoretical bounds applied to results	`day28_eval_report.pdf`
29-35	MUSHRA-style listening test with 8 raters	listener-grounded evaluation	`day35_listening_study.csv`
36-42	Final report write-up + code release	–	`final_report.pdf`

Sprint rule. If a day slips, drop the audio output to a shorter clip but never drop the audio entirely. The day-by-day audio trail *is* the project log.

2 Executive summary

Total fresh code budget: roughly 1500 lines of Python. Of the five pipeline stages, four reuse code I wrote during PS1–PS8; only Stage D (SparseNet) is a genuinely new implementation in PyTorch. No new mathematics is required beyond what ENGS 109 covered. Hardware: MacBook Pro M3 for Stages A–C and E; Dartmouth Discovery cluster (one A100) for Stage D. Day-0 RPCA already runs in 9.2 s on the MacBook with no GPU.

3 Repository layout

```
engs109-music-separation/
|-- README.md
|-- environment.yml
```

```
|-- pyproject.toml
|-- data/
|   |-- musdb18hq/              # Symlink to /scratch dataset.
|   `-- cache/                  # Pre-computed STFTs (.npz).
|-- src/separation/
|   |-- __init__.py
|   |-- io.py                   # WAV loading, MUSDB18 iterator.
|   |-- features.py             # STFT, mel, CQT, inverse.
|   |-- masks.py                # Mask <-> waveform helpers.
|   |-- metrics.py              # museval + mir_eval wrappers.
|   |-- stage_a_rpca.py         # Inexact-ALM RPCA (from PS8).
|   |-- stage_b_nmf.py          # NMF + class-conditional dictionaries.
|   |-- stage_c_ksvd.py         # K-SVD + SRC (from PS3, PS7).
|   |-- stage_d_scn.py          # SparseNet (new, PyTorch).
|   |-- stage_e_mmv.py          # SOMP for stereo (extends PS3 OMP).
|   `-- train.py                # Stage D training entry point.
|-- notebooks/
|   |-- 00_data_smoke_test.ipynb
|   |-- 01_stage_a_rpca.ipynb
|   |-- 02_stage_b_nmf.ipynb
|   |-- 03_stage_c_ksvd.ipynb
|   |-- 04_stage_d_scn_overfit.ipynb
|   |-- 05_evaluation.ipynb
|   `-- 06_listening_study.ipynb
|-- experiments/                # Configs in YAML.
|-- reports/                    # Final figures, .tex.
|-- tests/                      # Unit tests for solvers.
`-- audio_examples/             # Curated test clips for the report.
```

4 Environment setup

environment.yml

```yaml
name: engs109-mss
channels: [conda-forge, pytorch]
dependencies:
  - python=3.11
  - numpy=2.0
  - scipy=1.13
  - matplotlib=3.9
  - scikit-learn=1.5
  - librosa=0.10
  - soundfile=0.12
  - ffmpeg=6.1
  - pytorch=2.3
  - cvxpy=1.5            # used in PS7, kept for BP/LP baselines
  - jupyterlab=4.2
  - pip
  - pip:
      - musdb==0.4.0
      - museval==0.4.1
      - mir_eval==0.7
      - openunmix==1.2.1
      - demucs==4.0.1
```

The same file is checked into the repository so that `conda env create -f environment.yml` reconstructs everything bit-identically.

Hardware

Local dev: MacBook Pro M3, 36 GB unified memory; runs Stages A–C and all evaluation comfortably. Stage D training: Dartmouth Discovery cluster, one NVIDIA A100 (40 GB) for an estimated 40–60 wall-clock hours over the 10-week window.

5 Data pipeline

Dataset acquisition

MUSDB18-HQ ships as one `.tar.gz` (≈ 22 GB) at zenodo.org/record/3338373. Extract to `/scratch/musdb18hq/` (Discovery) or `$HOME/data/musdb18hq/` (local) and symlink to `data/musdb18hq/`. Standard splits: 100 training, 50 test songs. I will reserve 14 songs from train as validation (`musdb.DB(subsets="train", split="valid")`).

Feature extraction

Reuse the STFT helper I already wrote for `MozartServe/audio2midi/cqt.py`:

```python
# src/separation/features.py
import librosa
import numpy as np

SR, N_FFT, HOP, WIN = 22050, 2048, 512, 2048

def stft(y: np.ndarray) -> np.ndarray:
    """Complex STFT; rows = freq bins, cols = time frames."""
    return librosa.stft(y, n_fft=N_FFT, hop_length=HOP, win_length=WIN)

def istft(Y: np.ndarray) -> np.ndarray:
    return librosa.istft(Y, hop_length=HOP, win_length=WIN)

def magphase(Y):        # complex -> (magnitude, phase)
    return np.abs(Y), np.angle(Y)

def apply_mask(M: np.ndarray, Y: np.ndarray) -> np.ndarray:
    """Soft mask M in [0,1] on magnitude, keep mixture phase."""
    return istft(M * np.abs(Y) * np.exp(1j * np.angle(Y)))
```

For the SparseNet stage I additionally use mel-band spectrograms (64 mels) so input patches are small enough for FISTA inference to stay under 50 ms per frame.

Augmentation (training-only)

On-the-fly stem remixing: pick four random stems from different songs (`vocals`, `drums`, `bass`, `other`), apply a random gain $g \sim \mathcal{U}(-6, 6)$ dB to each, and sum. This is the standard SiSEC-2018 augmentation and roughly 20×'s the effective dataset size. Implemented in a custom `torch.utils.data.IterableDataset`.

6 Code reuse map

Table 1 keys every component back to existing problem-set code. I will refactor each PS notebook into a single library module so nothing gets re-derived from scratch.

Table 1: Existing problem-set code that becomes the spine of each stage.

Module	Where it comes from	What needs changing
OMP solver	PS3 `omp(A, y, S)`	Vectorise over time frames; add MMV variant for Stage E.
ISTA/FISTA	PS5 `fista(A, y, lam)`	Wrap as a `torch.autograd.Function` for Stage D unrolling.
Basis Pursuit / LASSO	PS7 `cvxpy` formulations	Only used as a sanity check vs FISTA.
SRC classifier	PS7 face-recognition notebook	Replace class labels with {vocal, instrumental}.
K-SVD	PS3 supplementary	Add a per-source training loop.
Inexact-ALM nuclear norm	PS8 matrix-completion notebook	Add the sparse $\boldsymbol{E}$ update for full RPCA.
NMF multiplicative updates	*New* (sklearn baseline first)	Implement KL-divergence variant from L15 slides.
Stage D SparseNet	*Fully new*, PyTorch	4 composite modules; see §7.4.
Evaluation harness	*New*, thin wrapper	Calls `museval.eval_mus_track`.

7 Per-stage implementation

7.1 Stage A. Robust PCA (4 days)

The full algorithm is the inexact ALM of Lin–Chen–Ma (2010), already implemented in the figure-generation script for the proposal and verified to recover the synthetic 60×80 rank-3 matrix to 10^{-7} in 32 iterations.

```python
# src/separation/stage_a_rpca.py
import numpy as np

def shrink(X, tau):
    return np.sign(X) * np.maximum(np.abs(X) - tau, 0.0)

def svt(X, tau):
    U, s, Vt = np.linalg.svd(X, full_matrices=False)
    s = np.maximum(s - tau, 0.0)
    return U @ np.diag(s) @ Vt

def rpca(Y, lam=None, tol=1e-7, max_iter=200):
    n1, n2 = Y.shape
    lam = 1.0 / np.sqrt(max(n1, n2)) if lam is None else lam
    norm_two = np.linalg.norm(Y, 2)
```

```
    Y2 = Y / max(norm_two, np.linalg.norm(Y, np.inf) / lam)
    mu, rho = 1.25 / norm_two, 1.5
    L, E = np.zeros_like(Y), np.zeros_like(Y)
    for k in range(max_iter):
        L = svt(Y - E + Y2 / mu, 1.0 / mu)
        E = shrink(Y - L + Y2 / mu, lam / mu)
        Y2 = Y2 + mu * (Y - L - E)
        mu = mu * rho
        if np.linalg.norm(Y - L - E, "fro") / np.linalg.norm(Y, "fro") < tol:
            break
    return L, E
```

Applied to magnitude spectrograms $\boldsymbol{M}_Y \in \mathbb{R}_{\geq 0}^{F \times T}$, L is the harmonic stem estimate, E the percussive one. Wiener masking gives soft masks: $\boldsymbol{M}_h = L^2/(L^2+E^2)$, $\boldsymbol{M}_p = E^2/(L^2+E^2)$.

7.2 Stage B. NMF with class-conditional dictionaries (5 days)

Training: separate Lee–Seung loops on vocal-only and instrumental-only training stems, $K_v = K_i = 64$, 400 iters, KL divergence. sklearn's `NMF(beta_loss="kullback-leibler", solver="mu")` works out of the box. Test-time decomposition holds $\boldsymbol{D}$ fixed and solves only for $\boldsymbol{H}$.

```
# src/separation/stage_b_nmf.py
from sklearn.decomposition import NMF
import numpy as np

def train_dictionary(M_train, K=64, max_iter=400):
    """M_train: (F, T_all) nonnegative magnitude spectrogram."""
    nmf = NMF(n_components=K, init="nndsvda", solver="mu",
              beta_loss="kullback-leibler", max_iter=max_iter)
    H = nmf.fit_transform(M_train.T).T   # H: (K, T_all)
    D = nmf.components_.T                # D: (F, K)
    return D, H

def separate(M_mix, D_v, D_i, beta=1e-2, max_iter=200):
    """Joint factorisation with concatenated dictionary."""
    F, T = M_mix.shape
    D = np.hstack([D_v, D_i])
    Kv, Ki = D_v.shape[1], D_i.shape[1]
    H = np.random.rand(Kv + Ki, T) + 1e-3
    for _ in range(max_iter):
        num = D.T @ M_mix
        den = D.T @ D @ H + beta + 1e-9
        H *= num / den
    Mv_hat = D_v @ H[:Kv]
    Mi_hat = D_i @ H[Kv:]
    mask_v = (Mv_hat ** 2) / (Mv_hat ** 2 + Mi_hat ** 2 + 1e-9)
    return mask_v
```

Output `mask_v` multiplies the complex STFT then inverts. Two ablations to run: (a) Frobenius vs KL loss, (b) $K \in \{32, 64, 128\}$.

7.3 Stage C. K-SVD + SRC (5 days)

Train one K-SVD dictionary per source from 4,096 randomly-sampled spectrogram frames each. Per-frame sparsity $S = 10$, dictionary size $K = 256$, 80 outer iterations. Implementation reuses my PS3 OMP and adds the column-by-column SVD update from L14:

```
# src/separation/stage_c_ksvd.py
import numpy as np

def omp(D, y, S):
    """Reused from PS3; returns S-sparse x with y ~= D x."""
    residual = y.copy()
    support, x = [], np.zeros(D.shape[1])
    for _ in range(S):
        idx = int(np.argmax(np.abs(D.T @ residual)))
        if idx in support:
            break
        support.append(idx)
        x_s, *_ = np.linalg.lstsq(D[:, support], y, rcond=None)
        x[:] = 0
        x[support] = x_s
        residual = y - D @ x
    return x

def ksvd(Y, K, S, n_iter=80):
```

```
    F, L = Y.shape
    D = Y[:, np.random.choice(L, K, replace=False)]
    D /= np.linalg.norm(D, axis=0, keepdims=True) + 1e-9
    X = np.zeros((K, L))
    for it in range(n_iter):
        for j in range(L):
            X[:, j] = omp(D, Y[:, j], S)
        for k in range(K):
            wk = np.flatnonzero(X[k])
            if len(wk) == 0:
                continue
            Ek = Y[:, wk] - D @ X[:, wk] + np.outer(D[:, k], X[k, wk])
            U, s, Vt = np.linalg.svd(Ek, full_matrices=False)
            D[:, k] = U[:, 0]
            X[k, wk] = s[0] * Vt[0]
    return D, X
```

At test time, build the union dictionary $\boldsymbol{D} = [\boldsymbol{D}_v \mid \boldsymbol{D}_i]$, run OMP per frame, then assign each non-zero atom to its source. Per-bin source assignment follows the L8 residual rule.

7.4 Stage D. SparseNet mask predictor (3 weeks)

The genuinely new code. The composite sparse coding module of L16 is implemented as a PyTorch `Module` whose forward pass unrolls $T_f = 30$ FISTA iterations for both the upsampling and downsampling sparse codes; gradients flow back via PyTorch autograd.

```
# src/separation/stage_d_scn.py  (sketch)
import torch
import torch.nn as nn

def fista_nn(D, y, lam1, lam2, T=30):
    """Non-negative FISTA for (1/2)||y - D a||^2 + lam1||a||_1 + (lam2/2)||a||_2^2."""
    L = torch.linalg.matrix_norm(D, ord=2) ** 2 + lam2
    a = torch.zeros(D.shape[1], device=D.device)
    z, t = a.clone(), 1.0
    for _ in range(T):
        grad = D.T @ (D @ z - y) + lam2 * z
        a_new = torch.clamp_min(z - grad / L - lam1 / L, 0.0)
        t_new = (1 + (1 + 4 * t ** 2) ** 0.5) / 2
        z = a_new + ((t - 1) / t_new) * (a_new - a)
        a, t = a_new, t_new
    return a

class CompositeModule(nn.Module):
    def __init__(self, n_in, n_up, n_down, lam1=0.05, lam2=0.05):
        super().__init__()
        self.D1 = nn.Parameter(torch.randn(n_in, n_up) * 0.05)
        self.D2 = nn.Parameter(torch.randn(n_up, n_down) * 0.05)
        self.lam1 = nn.Parameter(torch.tensor(lam1))
        self.lam2 = nn.Parameter(torch.tensor(lam2))
    def forward(self, x):                                  # x: (B, n_in)
        a1 = torch.stack([fista_nn(self.D1, xi, self.lam1, self.lam2)
                          for xi in x])
        a2 = torch.stack([fista_nn(self.D2, ai, self.lam1, self.lam2)
                          for ai in a1])
        return a2                                          # (B, n_down)

class SparseNet(nn.Module):
    def __init__(self, F=64, L=4, n_up=128, n_down=64):
        super().__init__()
        dims = [F] + [n_down] * L
        self.modules_ = nn.ModuleList(
            CompositeModule(dims[i], n_up, dims[i + 1]) for i in range(L))
        self.head = nn.Conv2d(n_down, 1, kernel_size=1)
    def forward(self, X):                                  # X: (B, F, T)
        h = X.transpose(1, 2).reshape(-1, X.shape[1])    # (B*T, F)
        for mod in self.modules_:
            h = mod(h)
        h = h.reshape(X.shape[0], X.shape[2], -1).transpose(1, 2)
        h = h.unsqueeze(1)                                  # (B, 1, n_down, T)
        return torch.sigmoid(self.head(h)).squeeze(1)  # (B, F, T) mask
```

Two implementation details that are not optional:

1. **Batch the FISTA.** The naive per-sample loop in `forward` above is for readability; the production code batches into a single (B, N_u) matmul per iteration so a 30-step inference is one fused kernel call.

2. **Pre-compute Lipschitz constants.** L16 explicitly notes "Matrix inversion is pre-computed." At each gradient step I cache $\|\boldsymbol{D}^{(\ell)}\|_2^2$ via power iteration once per epoch rather than recomputing each forward pass.

Training loop.

```python
# src/separation/train.py
import torch
from separation.stage_d_scn import SparseNet
from separation.features import stft, magphase

net = SparseNet(F=64, L=4).cuda()
opt = torch.optim.AdamW(net.parameters(), lr=3e-4, weight_decay=1e-5)

for epoch in range(100):
    for mix_wav, voc_wav in loader:
        Ymix, Yvoc = stft(mix_wav), stft(voc_wav)
        Mmix, Pmix = magphase(Ymix)
        Mvoc, _    = magphase(Yvoc)
        mask = net(Mmix.cuda())
        recon = mask * torch.as_tensor(Mmix).cuda()
        loss = (recon - torch.as_tensor(Mvoc).cuda()).abs().mean()
        loss = loss + 1e-3 * sum(p.abs().mean() for p in net.parameters()
                                 if p.requires_grad)
        opt.zero_grad(); loss.backward(); opt.step()
```

Hyperparameters (fixed at the start, then frozen). L_{f}=30 FISTA steps, $\lambda_1 = \lambda_2 = 0.05$ initial, 4 composite modules, fat dictionary width $N_u = 128$, tall dictionary width $N_d = 64$, batch $B = 16$ song-snippets of 4 s, learning rate 3×10^{-4} AdamW with cosine decay, 100 epochs. Mixed precision (`torch.amp`) on the A100.

7.5 Stage E. SOMP for stereo (3 days)

The simultaneous OMP of L12 needs only a one-line change to the PS3 OMP: the scoring step becomes a row-norm of the residual matrix.

```python
def somp(D, Y, S, q=2):
    """Y: (F, K_meas).  Returns row-sparse X: (atoms, K_meas)."""
    R = Y.copy()
    support = []
    for _ in range(S):
        scores = np.linalg.norm(D.T @ R, ord=q, axis=1)
        idx = int(np.argmax(scores))
        if idx in support:
            break
        support.append(idx)
        Xs, *_ = np.linalg.lstsq(D[:, support], Y, rcond=None)
        R = Y - D[:, support] @ Xs
    X = np.zeros((D.shape[1], Y.shape[1]))
    X[support] = Xs
    return X
```

At inference, stack $\boldsymbol{y}_L, \boldsymbol{y}_R$ as the 2-column $\boldsymbol{Y}$ and call `somp(D, Y, S=10)`. The shared row support is the joint sparse code; per-channel reconstructions then go through the standard inverse STFT with their own phases.

8 Training schedule

Stage	Training data	Wall-clock estimate
A. RPCA	none (no training)	instantaneous
B. NMF	1000 vocal + 1000 instr. frames	8 min CPU
C. K-SVD	4096 frames per source	25 min CPU
D. SparseNet	MUSDB18 train, augmented	45–60 h A100
E. SOMP	none (uses Stage C dict.)	instantaneous

9 Evaluation harness

```python
# src/separation/metrics.py
import museval, musdb
def evaluate_on_musdb(separator_fn, output_dir):
    db = musdb.DB(root="data/musdb18hq", subsets=["test"])
    results = museval.EvalStore()
    for track in db:
        estimates = separator_fn(track.audio, track.rate)
        scores = museval.eval_mus_track(track, estimates,
```

```
                                        output_dir=output_dir)
        results.add_track(scores)
    return results
```

`separator_fn` returns a dict `{"vocals": ndarray, "accompaniment": ndarray}` at the original sample rate. Reports median SDR/SI-SDR/SIR/SAR per stem with 25th–75th percentile bars, matching the SiSEC convention.

10 Notebook plan (mirroring PS format)

Each notebook follows the PS template I used all term: a header cell with the problem statement, math derivation in markdown, a single solver function, a synthetic verification block, then the real-data block.

- **00_data_smoke_test.ipynb**: confirm MUSDB18-HQ loads, plot the first track's STFT/CQT/mel of each stem.
- **01_stage_a_rpca.ipynb**: synthetic verification (as in the proposal Fig. 2), then apply to a MUSDB track; render X/E spectrograms; tabulate SDR.
- **02_stage_b_nmf.ipynb**: train $\boldsymbol{D}_v$ and $\boldsymbol{D}_i$ on training stems; show learned atoms; report SDR with KL vs Frobenius.
- **03_stage_c_ksvd.ipynb**: K-SVD training on synthetic sparse signals first (sanity), then on spectrogram frames; SRC residual visualisation.
- **04_stage_d_scn_overfit.ipynb**: deliberately overfit SparseNet to a single MUSDB track to verify backward pass; check that intermediate codes are actually sparse ($>95\%$ zeros).
- **05_evaluation.ipynb**: run all five stages on the 50-song test set; produce the headline SDR/SI-SDR boxplots and the per-track CSVs.
- **06_listening_study.ipynb**: pick 20 clips, anonymise, collect MUSHRA scores from 8 raters; box-plot the results.

11 Risk register and fallbacks

Risk	Detection	Fallback
SparseNet too slow per iter	>200 ms/frame on A100	Drop to $L=2$ composite modules, replace mel input with the first 32 NMF activations from Stage B (effectively shrinks n_{in} from 64 to 32).
SparseNet does not converge	validation SDR flat after 30 epochs	Warm-start $\boldsymbol{D}^{(1)}$ from K-SVD-trained dictionary (Stage C); linearly schedule $\lambda_1 : 0.5 \to 0.05$.
MUSDB-HQ ground-truth alignment drifts	Per-track SDR has bimodal distribution	Run `museval` alignment compensation; document as known issue.
Listening-study response rate low	<4 raters in week 9	Reduce sample count to 10 clips; use myself + 2 Mozart AI teammates as fallback raters.
Discovery cluster queue blocks training	no GPU available for >48 h	Train Stage D locally on M3 with reduced model: $L=2$, batch $=4$, 15 epochs (about 20 wall-clock hours).

12 Appendix A: Expected figure list

For the final report (separate document from this plan):

- Fig. 1: pipeline diagram (already produced for proposal).
- Fig. 2: spectrogram triptych mix/vocals/instrumental (already produced).
- Fig. 3: chromagram + harmonic–percussive split for one representative MUSDB song.
- Fig. 4: RPCA decomposition of spectrogram with quantified SDR per stage.
- Fig. 5: NMF learned atoms $\boldsymbol{D}_v, \boldsymbol{D}_i$ as small spectral heatmaps (analogue of the SCN feature-map figure on L16 slide 14).
- Fig. 6: K-SVD atoms for both sources, with per-atom sparsity histograms.
- Fig. 7: SparseNet intermediate sparsity profile; demonstrating the L16 "$>95\%$ zeros" regime.
- Fig. 8: SDR boxplots on MUSDB18-HQ test set, all stages and baselines.
- Fig. 9: MUSHRA listening-study results.
- Fig. 10: ablation table on L, K, S, λ_1.

13 Appendix B: Connection to my Mozart AI codebase

What I will lift directly:

- `MozartServe/audio2midi/cqt.py` provides constant-Q-transform helpers that I will reuse for the CQT variant of Stage B.
- `MozartServe/audio2midi/midi.py` for ground-truth onset/offset tagging when I want to qualify separation quality on monophonic instrumental excerpts.
- Mozart AI's user-track manifest format (one JSON per project) gives me a ready-made schema for storing the listening-study trials.

What goes back into Mozart AI: If Stage D outperforms ElevenLabs on vocal SDR (the success criterion), I drop the trained checkpoint into a new `MozartServe/separation/` package and replace the ElevenLabs HTTP call in the "Extract Stems" button. This closes the 35 dB SI-SDR gap from my MS thesis and removes one paid third-party dependency from the product.

14 Appendix C: What I am explicitly *not* doing

To keep this honest about scope:

- No multi-source separation beyond vocals vs accompaniment. 4-stem (vocals/drums/bass/other) is a stretch goal only.
- No time-domain models (Wave-U-Net, Conv-TasNet). The whole point of the project is to stay in the spectrogram CS regime.
- No transformer architectures. L16's SparseNet is the deep architecture the course gave us; that is the one I am evaluating.
- No real-time inference target. Sample-level offline batch is the only operating point.
- No additional datasets beyond MUSDB18-HQ. Cross-dataset generalisation is left for future work.